SIMATS ENGINEERING — CSA37: Software Testing

Assessment Tool 1 (AT1): Descriptive Written Exam Solutions | CO1 Solutions

Q1: Unstructured Testing in Banking Application

Answer: Ad-hoc testing leads to missed defects due to lack of systematic planning. Treating testing as an engineering activity involves disciplined test design using boundary value analysis, equivalence partitioning, and standardized tools. Structured test design ensures predictable test coverage, reproducible test scripts, and systematic defect tracking, significantly improving defect detection and software quality assurance.

Q2: ATM System and Structured Testing Process across SDLC

Answer: Implementing a structured testing process across SDLC phases (Requirement, Design, Coding, Testing, Deployment) ensures early defect discovery. For ATM withdrawal functionality, requirement reviews check transaction limits, design reviews validate security logic, unit testing checks pin verification, and system testing validates end-to-end cash dispensing. Process consistency eliminates ambiguity and drastically reduces defect leakage.

Q3: Error, Defect, and Failure in Login & Transactions

Answer: An *Error* is a human mistake (e.g., programmer misinterpreting session timeout rules). A *Defect* (bug) is the manifestation of that error in code (e.g., incorrect conditional check). A *Failure* is the observable incorrect behavior during execution (e.g., login success message displayed but dashboard fails to load). Understanding these terms improves root cause analysis and clarity in defect reporting.

Q4: Defect Clustering & Pareto Principle in Test Planning

Answer: Defect clustering (Pareto Principle) states that a small number of modules contain the majority of defects (80/20 rule). Testers should prioritize high-risk, complex modules (e.g., payment gateways) during test planning. Focusing effort where defects are concentrated optimizes testing efficiency, reduces test execution time, and enhances overall product reliability.

Q5: Defect Repository & Prevention of Recurring Defects

Answer: A defect repository stores historical defect data including root causes, severity, and module origin. Analyzing repository data guides targeted test case design for future releases. By identifying recurring defect patterns (e.g., SQL injection vulnerabilities in forms), teams can implement proactive code reviews and developer training to prevent recurrence.

Q6: Mobile Banking Fund Transfer Crashes

Answer: Mobile banking crashes during fund transfers typically stem from unhandled edge cases, concurrency issues, or memory leaks. Systematic testing principles—such as exhaustive testing is impossible, early testing, and defect clustering—guide structured boundary condition checks and load simulation, ensuring robust exception handling and stable mobile operations.

Q7: Differentiating Testing and Debugging

Answer: *Testing* is the process of executing software to identify failures and discrepancies between expected and actual behavior. *Debugging* is the development activity of locating, analyzing, and fixing the root cause of identified defects in the source code. Testing proves failures exist; debugging finds and fixes the code defect. Integrating both structured test execution with systematic developer debugging ensures efficient quality delivery.

Q8: Cross-Browser Testing Strategy for E-Commerce Platforms

Answer: E-commerce platforms experience cross-browser inconsistencies due to variations in rendering engines (WebKit, Blink, Gecko), CSS support, and JavaScript execution. A structured cross-browser testing strategy utilizes automated browser testing tools (e.g., Selenium, Playwright), responsive design validations, and standard conformance testing to guarantee uniform user experience and reliability.

Q9: Payroll System Defect Classification and Prevention

Answer: Payroll calculation errors are classified under logical, algorithmic, or computational defect categories. Understanding defect classes helps engineers design specialized test suites (e.g., boundary tax brackets, overtime multipliers). Preventing these involves rigorous unit testing, peer code reviews, and automated calculation verification against standard financial test vectors.

Q10: Engineering Approach vs. Random Testing

Answer: Random testing lacks structured traceability and often misses critical logical paths. Treating testing as an engineering activity involves systematic test case design techniques—such as Equivalence Partitioning, Boundary Value Analysis, and Decision Table testing. This structured approach guarantees systematic coverage, traceability to requirements, and high defect detection efficiency.

Q11: Healthcare Application Data Inconsistency

Answer: Data inconsistency across healthcare modules originates from poor interface validation and lack of integrated database constraints. Systematic testing principles, such as integration testing and rigorous data validation protocols, verify data integrity across modules. Clear defect tracking and standardized reporting ensure developers address synchronization bottlenecks early.

Q12: Using Defect Repositories for Pattern Identification

Answer: Persistent defects occur when organizations fail to analyze past bug repositories. Repository data allows QA teams to extract historical defect trends, identify error-prone modules, and construct predictive test cases. Implementing repository-driven metrics prevents recurring bugs and optimizes future test planning.

Q13: Web-Based Learning System Performance Under Peak Load

Answer: Systems fail during peak usage due to missing non-functional performance and stress testing scenarios. Applying software testing principles means planning comprehensive load, stress, and scalability testing early. Better test strategies simulate peak concurrent user loads, ensuring high system reliability and preventing service outages.

Q14: Early Involvement of Testers in SDLC

Answer: Late tester involvement leads to undetected requirement ambiguities. Involving testers during the requirement analysis phase enables early defect prevention (shifting left), validation of testability, and creation of acceptance criteria. Collaboration between developers and testers from inception drastically reduces rectification costs and improves quality.

Q15: Improving Test Coverage for Complex Software Products

Answer: Incomplete test coverage leads to intermittent production failures. Applying testing principles—such as complete testing is impossible, requiring risk-based testing—helps prioritize critical paths. Improving test case design using combinatorial testing and state transition models ensures robust validation and reliable software performance.

Q16: Logistics Application Defect Reporting and Tracking

Answer: Poor defect reporting causes delays in issue identification. Structured defect reports containing precise steps to reproduce, expected vs. actual outcomes, environment details, and severity levels enhance communication between testers and developers. Standardized reporting accelerates triage and resolution time.

Q17: Process-Based Testing Approaches for Module Consistency

Answer: Inconsistent testing across modules creates uneven quality. Standardized process-based testing frameworks (e.g., TMMi or ISO/IEC standards) establish uniform guidelines for test planning, execution, and review. Standardization ensures repeatable, high-quality outcomes across all project modules.

Q18: Post-Deployment Failures and Boundary Testing

Answer: Post-deployment failures often occur because testing focused only on nominal inputs, ignoring boundary and edge cases. Applying boundary value analysis (BVA) ensures that extreme valid and invalid limits (min, min-1, max, max+1) are rigorously tested, preventing unexpected crashes and edge-case exceptions.

Q19: Defect Documentation and Repository Management

Answer: Lack of proper defect documentation hinders trend analysis. Implementing structured defect repositories with categorization by root cause, module, and severity improves tracking, enables statistical quality control, and provides valuable metrics for continuous process improvement.

Q20: Redesigning Banking Transaction Testing Approach

Answer: Repeated transaction failures require a complete redesign of the testing approach toward structured engineering practices. This involves introducing automated test frameworks, rigorous requirement traceability matrices, formal test reviews, and comprehensive boundary and exception testing to ensure absolute financial reliability.